DevInterview AI

Product Requirements Document

Interview Support, Technical Notebook, and English Learning Platform for Software Developers

Product Direction

A technical interview preparation workspace that combines senior-level technical theory notes, AI mock interviews, interview-connected English feedback, and personalized learning recommendations based on the user learning profile.

Document Item	Value
Version	2.0 - Updated PRD
Audience	Founder, Product, Engineering, Design, AI/ML, QA
Primary Users	Software developers preparing for interviews in English
Core Loop	Learn theory -> Practice interview -> Get feedback -> Save weak concepts -> Improve English and technical notes -> Practice again

1. Executive Summary

DevInterview AI is a web application for software developers who want to prepare for technical interviews while improving English communication. The application is not only a mock interview tool. It is a learning workspace where users build structured technical theory notes, practice interview questions, receive technical and English feedback, and get personalized next-step recommendations.

The key product insight is that interview readiness requires three connected capabilities: strong technical theory, the ability to explain that theory under interview pressure, and enough English fluency to sound clear and professional. DevInterview AI connects those capabilities into one feedback loop.

The updated PRD introduces three major product decisions: onboarding-based personalization defaults, a Smart Technical Notebook for structured theory, and an interview-connected English Notebook that captures short grammar and naturalness improvements from real answers.

Product Pillar	Description	Why It Matters
Smart Technical Notebook	Users paste rough notes and AI restructures them into senior-level technical notes.	Creates a reliable theory base before interview practice.
AI Mock Interview	AI asks role-based questions, evaluates answers, and recommends next questions.	Simulates real interview pressure and exposes weak areas.
English Feedback Notebook	AI checks grammar, clarity, naturalness, and saves short English notes from real answers.	Improves interview communication with contextual English practice.
Recommendation Engine	AI recommends next questions, notebook topics, and English topics based on role and weak areas.	Keeps learning personalized and continuous.

2. Problem Statement

Developers preparing for international or remote interviews often have fragmented preparation. They may study technical concepts in one place, practice English in another place, and answer mock interview questions somewhere else. This creates a disconnected learning experience.

Common problems include messy technical notes, weak interview answer structure, difficulty explaining production tradeoffs, limited feedback on grammar and naturalness, and no clear recommendation on what to study next after a poor answer.

- Developers know syntax but struggle to explain architecture, tradeoffs, cost, scaling, security, and failure modes.
- English learning is often generic and disconnected from real technical interviews.
- Mock interview tools often ask questions but do not build a long-term learning memory.
- Users repeatedly need to provide role, level, stack, and goals unless the product has a shared learning profile.
- Question banks alone do not identify why a user is weak or what they should study next.

3. Goals and Non-Goals

Goals	Description
Reduce setup friction	Collect role, level, English goal, and tech stack once during onboarding and reuse them across modules.

Improve technical theory	Convert rough notes into structured, senior-level technical notes.
Improve interview answering	Ask role-based questions and evaluate answer quality.
Improve English from real usage	Detect grammar and naturalness issues from actual interview answers.
Recommend next learning	Suggest next questions, theory notes, and English topics based on weak concepts.

Non-Goals	Description
Not a generic English course	English learning should be contextual to developer interviews.
Not only LeetCode	Coding questions may exist later, but the first product focuses on explanation, theory, and communication.
Not a passive note app	Notes must connect to interview questions, weak concepts, and learning recommendations.

4. Target Users and Personas

Persona	Profile	Primary Need
International Fullstack Developer	2-5 years experience, wants remote jobs, uses React, Next.js, Node.js, NestJS, PostgreSQL.	Prepare senior-level technical answers and explain clearly in English.
Backend Developer	Works with APIs, databases, authentication, queues, caching, system design.	Build production-focused theory and practice architecture tradeoffs.
Frontend Developer	Works with React, Next.js, TypeScript, browser performance, UX, testing.	Explain frontend concepts and performance topics clearly.
Career Switcher / Junior Developer	Has basic syntax knowledge but lacks interview structure.	Learn concepts, build confidence, and improve English foundation.
Bootcamp / Team User	Organization wants structured interview prep for cohorts.	Track progress and standardize learning content.

5. Product Learning Loop

Core Loop

Technical Note -> Interview Question -> User Answer -> Technical Feedback -> English Feedback -> Save Weak Concepts -> Save Short English Notes -> Recommend Next Question and Learning Topic.

Step	System Action	User Value
Onboarding	Collect learning profile once.	Personalized defaults without repeated setup.
Theory	Generate structured notes from rough input.	Clear technical foundation.
Practice	Ask role-based questions from	Real interview simulation.

	profile or notebook.	
Feedback	Evaluate technical correctness, answer structure, English grammar, naturalness.	Actionable improvement.
Memory	Store questions, answers, weak concepts, English notes.	Long-term learning history.
Recommendation	Suggest next question, notebook topic, and English topic.	Clear next step.

6. Onboarding and Shared Learning Profile

The application must not repeatedly ask the user for role, depth, stack, and goals during note creation or interview sessions. These should be collected during first onboarding and saved as a reusable learning profile. All AI modules should use this profile by default.

Onboarding Field	Examples	Used By
Target role	Frontend Developer, Backend Developer, Fullstack Developer, DevOps Engineer	Notebook, interviews, recommendations, resume analyzer
Current level	Junior, Mid-level, Senior, Staff	Learning path and difficulty calibration
Target interview level	Mid-level, Senior, Staff	AI output depth and question difficulty
Tech stack	React, Next.js, NestJS, PostgreSQL, Redis	Code examples, notes, questions
English level	Beginner, Intermediate, Upper-intermediate, Advanced	English feedback depth and correction style
Interview goal	Remote jobs, senior promotion, system design, English speaking	Roadmap and recommendations
Preferred output style	Pragmatic, deep technical, simple English, interview-focused	AI note and feedback tone

```
interface UserLearningProfile {
  targetRole: string;
  currentLevel: 'junior' | 'mid' | 'senior' | 'staff';
  targetLevel: 'junior' | 'mid' | 'senior' | 'staff';
  englishLevel: 'beginner' | 'intermediate' | 'upper_intermediate' | 'advanced';
  techStack: string[];
  interviewGoal: string[];
  preferredOutputStyle: 'pragmatic' | 'deep_technical' | 'simple_english' | 'interview_focused';
  preferredCodeLanguage: 'typescript';
  preferredBackendFramework?: 'nestjs' | 'express';
  preferredDatabase?: 'postgresql' | 'mysql' | 'mongodb';
}
```

Product Rule

Collect role, level, stack, English level, and goals during onboarding. Reuse them everywhere. Allow updates in Settings and per-note or per-session overrides through Advanced Settings only.

7. Smart Technical Notebook

The Smart Technical Notebook allows users to create technical learning notes from rough notes. AI refactors and expands the input into a standardized, production-focused technical note suitable for interview preparation. The note format emphasizes architecture, data boundaries, cost, scaling, security, operational reliability, debugging, and tradeoffs.

7.1 Simplified Note Creation Flow

1. User opens Technical Notebook.
2. User clicks New Note.
3. User enters topic, for example JWT Authentication.
4. User pastes rough notes.
5. App generates the note using saved onboarding defaults.
6. User can optionally open Advanced Settings to override role, depth, tech stack, or output style for this note only.

UI Element	Default Behavior	Advanced Override
Topic	Required input from user	N/A
Rough notes	Optional but recommended	N/A
Target role	Loaded from learning profile	Override per note
Depth	Loaded from target interview level	Override per note
Tech stack	Loaded from learning profile	Override per note
Output style	Loaded from AI output preferences	Override per note

7.2 Technical Note Required Format

Section	Requirement
Purpose	Three paragraphs: what the topic covers, why it matters for senior engineers, and what a senior engineer should know how to do.
Quick Reference	Table with Concept, What It Answers, Production Importance.
Core Concepts	3-5 subtopics with definition, use case, anti-pattern, code example, tradeoff, misconception.
Mental Model	Simple analogy or rule of thumb.
Production Usage	Table with Use Case, Architecture Design, Access/Usage Pattern, Senior Concern.
Practical Examples	2-3 advanced snippets using TypeScript, NestJS, and SQL where relevant.
Common Pitfalls	Table with Pitfall, Why It Is Dangerous, Better Approach.
Debugging Checklist	10-15 production debugging checks.
Production Checklist	10-15 architecture, security, and scaling checks before deployment.
Senior Interview Signals	5-8 senior-level signals and one strong architect-level interview answer.

7.3 AI Technical Note Generator Prompt

```
Role:
Act as a Staff/Senior Fullstack Engineer creating a standardized technical capability document.

Task:
The user provides rough notes on a technical subject. Refactor and expand these notes into a
highly structured, production-focused Technical Note.

Tone and guidelines:
- Authoritative and pragmatic.
- Focus on production realities: cost, scaling, security, consistency, observability,
maintainability.
- Tradeoff-aware: explain why we do something and what we give up.
- Senior perspective: assume the reader knows basic syntax.
- Focus on architecture, data boundaries, failure modes, and operational reliability.
- Use TypeScript, NestJS, and SQL for code examples when relevant.

Required structure:
1. Purpose
2. Quick Reference
3. Core Concepts
4. Mental Model
5. Production Usage
6. Practical Examples
7. Common Pitfalls
8. Debugging Checklist
9. Production Checklist
10. Senior Interview Signals
```

7.4 Note Types

Note Type	Purpose	Example
Technical Theory Note	Deep structured concept knowledge.	JWT Authentication, Database Indexing, Redis Caching
System Design Note	Architecture-level design and production concerns.	Notification System, Rate Limiter, Chat System
Interview Answer Note	Prepare strong answer for a specific question.	How does JWT authentication work?
English Explanation Note	Help user explain a technical concept clearly in English.	Explain event loop in simple English

8. Interview Question Notebook and Practice History

Every interview question and answer should be stored. The product should remember what the user practiced, how they answered, what weak concepts were detected, what English issues appeared, and what was recommended next.

Captured Data	Description
Question	AI-generated or user-selected interview question.
Question source	Role-based, notebook-generated, weak-area recommendation, or manual.
Answer	Text answer, transcript, and later optional audio.
Technical feedback	Correctness, missing concepts, production concerns,

	better answer.
English feedback	Grammar, naturalness, clarity, professional tone.
Next questions	Follow-up, gap-filling, difficulty progression, English practice.
Learning recommendations	Notebook topics and English topics to review.

```
interface InterviewFeedback {
  overallScore: number;
  technicalScore: number;
  englishScore: number;
  clarityScore: number;
  strengths: string[];
  weaknesses: string[];
  correctedAnswer: string;
  betterAnswerExample: string;
  missingConcepts: string[];
  recommendedTopics: string[];
  nextQuestions: {
    question: string;
    reason: string;
    difficulty: 'easy' | 'medium' | 'hard' | 'senior';
    type: 'follow_up' | 'gap_filling' | 'role_based' | 'english_practice';
  }[];
}
```

9. Next Question and Learning Recommendation Engine

After each answer, the AI should recommend the next best question and next learning topic. The recommendation must use the user learning profile, interview history, note readiness, weak technical concepts, repeated English issues, and target interview level.

Recommendation Type	When Used	Example
Follow-up question	User answer is incomplete or shallow.	Can you explain a real case where useMemo improves performance?
Gap-filling question	User missed an important concept.	What is the difference between clustered and non-clustered indexes?
Difficulty progression	User answered well and should move deeper.	How would you design a rate-limited API for 1 million users?
English practice question	Technical answer is correct but unclear in English.	Explain event loop again using shorter sentences and one example.
Notebook topic recommendation	User lacks theory foundation.	Study Redis Token Blacklist Strategy before another JWT revocation question.

```
interface RecommendationContext {
  userProfile: UserLearningProfile;
  recentQuestions: string[];
  weakTechnicalConcepts: string[];
  weakEnglishTopics: string[];
  masteredTopics: string[];
  targetInterviewDate?: string;
  noteReadinessScores: Record<string, number>;
```

```
}
```

10. Interview-Connected English Feedback Notebook

English learning must be connected to real interview practice. The system should not teach random grammar lessons. It should detect grammar, clarity, and naturalness issues from the user actual interview answers, save only useful short notes, and recommend targeted English topics that improve interview communication.

10.1 English Feedback Flow

7. User answers an interview question.
8. AI evaluates technical correctness and English communication quality.
9. AI detects grammar, naturalness, clarity, professional tone, and answer structure issues.
10. System creates a short English note only when the issue is important, repeated, or interview-impacting.
11. System updates weak English topics and recommends focused practice.

Example English Note

Problem: You said "JWT is use for authentication." Better: "JWT is used for authentication." Why: This is passive voice. Use is/are + past participle. Practice: Redis is used for caching. Tokens are stored in cookies. Requests are validated by middleware. Study: Passive Voice and Present Simple.

```
interface InterviewEnglishFeedback {
  grammarScore: number;
  naturalnessScore: number;
  clarityScore: number;
  professionalToneScore: number;
  originalSentence: string;
  correctedSentence: string;
  naturalVersion: string;
  detectedIssues: {
    issue: string;
    exampleFromUser: string;
    correction: string;
    explanation: string;
    grammarTopic: string;
  }[];
  recommendedEnglishTopics: string[];
  shortNote: string;
}
```

10.2 English Note Types

Type	Purpose	Example
Grammar Note	Capture grammar patterns from real mistakes.	Passive voice: is used, are stored, is validated.
Natural Speaking Note	Make answers sound more professional and fluent.	Use client instead of repeating user.
Technical Vocabulary Note	Improve precise technical phrasing.	Attach the token to the Authorization header.
Answer Structure Note	Improve explanation organization.	Definition -> flow -> security concern -> tradeoff.

10.3 Noise Control Rule

The English Notebook should not save every typo or small issue. It should save only issues that are repeated, important, or damaging to interview communication.

Save	Do Not Save
Repeated passive voice mistake	One-time typo
Wrong tense that affects clarity	Minor punctuation issue
Unnatural technical phrasing	Stylistic preference with no interview impact
Poor answer structure	Small spelling mistake in a long answer

11. Resume Analyzer

The Resume Analyzer is a supporting module that helps users align their resume with target roles. It should use the same learning profile to tailor feedback. Resume review should focus on ATS compatibility, impact bullets, technical keyword relevance, clarity, grammar, and interview question generation from resume claims.

Feature	Description
Resume upload	PDF and DOCX support.
ATS analysis	Keyword and formatting checks.
Bullet improvement	Rewrite experience with stronger impact, metrics, and clarity.
Role fit	Compare resume to target role and stack.
Interview question generation	Generate questions from resume projects and claims.
English cleanup	Improve grammar and professional tone.

12. MVP Scope

Module	MVP Features
Onboarding and Settings	Collect learning profile once, update preferences in Settings, allow per-note and per-session advanced overrides.
Smart Technical Notebook	Create note, paste rough notes, generate structured senior-level note, save/edit note, organize by role/topic, generate interview questions from note.
AI Mock Interview	Select topic or note, answer by text, receive technical feedback, save question and answer, suggest next questions.
English Feedback Notebook	Detect grammar/naturalness/clarity issues from interview answers, save short notes, track weak English topics.
Learning Recommendation Engine	Recommend next question, notebook topic, and English topic based on weak concepts and profile.
Dashboard	Show progress, weak areas, note readiness, interview history, English weak topics.

Not MVP / Later	Description
Voice interview mode	Speech-to-text, audio recording, pronunciation, confidence scoring.
AI avatar	Real-time avatar interviewer with interruption simulation.
Coding environment	Live coding and code explanation evaluation.
Company-specific simulation	Google, Amazon, startup, remote job interview styles.
Team/bootcamp workspace	Admin dashboards and cohort progress.

13. Information Architecture and Navigation

```
Dashboard
Notebook
  - All Notes
  - Create Note
  - Technical Notes
  - System Design Notes
  - English Notes
Interview Practice
  - Start Mock Interview
  - Practice From Notebook
  - Question History
Learning Path
  - Recommended Topics
  - Weak Technical Concepts
  - Weak English Topics
Resume Analyzer
Progress
Settings
  - Learning Profile
  - AI Output Preferences
  - English Preferences
```

Screen	Primary Actions
Dashboard	Continue practice, view weak areas, open next recommendation.
Create Note	Enter topic, paste rough notes, generate note using profile defaults.
Note Detail	Edit note, regenerate section, generate questions, start practice.
Interview Session	Answer question, view feedback, choose next question.
English Notes	Review short grammar/naturalness notes and practice patterns.
Learning Path	View prioritized technical and English topics.
Settings	Adjust defaults for role, level, stack, English level, output style.

14. System Architecture

Layer	Recommended Stack	Responsibilities

Frontend	Next.js App Router, TypeScript, TailwindCSS, shadcn/ui	Notebook editor, interview UI, dashboard, settings, audio later.
Backend API	NestJS, TypeScript	Auth, profiles, notes, interviews, recommendations, AI orchestration.
Database	PostgreSQL	Users, notes, questions, answers, feedback, recommendations, weak concepts.
Cache / Queue	Redis, BullMQ	AI jobs, note generation, async scoring, rate limiting.
Object Storage	S3-compatible storage	Resume files, audio recordings later.
AI Services	LLM, speech-to-text later, text-to-speech later	Note generation, answer evaluation, English feedback, recommendations.
Observability	OpenTelemetry, logs, metrics, tracing	Monitor AI cost, latency, failures, user conversion.

```
function buildAIContext(params: {
  profile: UserLearningProfile;
  module: 'notebook' | 'interview' | 'resume' | 'learning_path';
  overrides?: Partial<UserLearningProfile>;
  currentTopic?: string;
  recentWeakAreas?: string[];
}) {
  return {
    userProfile: {
      ...params.profile,
      ...params.overrides,
    },
    currentModule: params.module,
    currentTopic: params.currentTopic,
    recentWeakAreas: params.recentWeakAreas ?? [],
  };
}
```

15. Data Model

15.1 Core Tables

```
CREATE TABLE user_learning_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL UNIQUE,
  target_role TEXT NOT NULL,
  current_level TEXT NOT NULL,
  target_level TEXT NOT NULL,
  english_level TEXT NOT NULL,
  tech_stack TEXT[] DEFAULT '{}',
  interview_goals TEXT[] DEFAULT '{}',
  preferred_output_style TEXT DEFAULT 'interview_focused',
  preferred_code_language TEXT DEFAULT 'typescript',
  preferred_backend_framework TEXT DEFAULT 'nestjs',
  preferred_database TEXT DEFAULT 'postgresql',
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE technical_notes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  user_id UUID NOT NULL,
  title TEXT NOT NULL,
  slug TEXT NOT NULL,
  target_role TEXT,
  experience_level TEXT,
  note_type TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'draft',
  raw_notes TEXT,
  structured_content JSONB NOT NULL,
  tags TEXT[],
  tech_stack TEXT[],
  readiness_score INT DEFAULT 0,
  interview_readiness_score INT DEFAULT 0,
  english_explanation_score INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

15.2 Interview Tables

```
CREATE TABLE interview_questions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  note_id UUID NULL REFERENCES technical_notes(id) ON DELETE SET NULL,
  session_id UUID,
  role TEXT NOT NULL,
  category TEXT NOT NULL,
  difficulty TEXT NOT NULL,
  question TEXT NOT NULL,
  question_type TEXT,
  expected_concepts TEXT[],
  source_section_key TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE interview_answers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  question_id UUID NOT NULL REFERENCES interview_questions(id),
  user_id UUID NOT NULL,
  answer_text TEXT,
  answer_audio_url TEXT,
  transcript TEXT,
  technical_score INT,
  english_score INT,
  clarity_score INT,
  overall_score INT,
  ai_feedback JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);
```

15.3 English Tables

```
CREATE TABLE english_notes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  source_answer_id UUID REFERENCES interview_answers(id) ON DELETE SET NULL,
  source_question_id UUID REFERENCES interview_questions(id) ON DELETE SET NULL,
  note_type TEXT NOT NULL,
  topic TEXT NOT NULL,
  user_sentence TEXT NOT NULL,
  corrected_sentence TEXT NOT NULL,
  natural_sentence TEXT,
```

```
  explanation TEXT,
  practice_patterns TEXT[],
  recommended_topics TEXT[],
  status TEXT DEFAULT 'needs_practice',
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE user_english_weaknesses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL,
  topic TEXT NOT NULL,
  weakness_score INT DEFAULT 0,
  examples_count INT DEFAULT 0,
  last_example TEXT,
  last_detected_at TIMESTAMP DEFAULT NOW(),
  status TEXT DEFAULT 'active'
);
```

16. AI Prompt Architecture

Prompt	Purpose	Output
Note Structuring Prompt	Convert rough notes into standardized technical note.	Structured note JSON and markdown.
Note Improvement Prompt	Enhance existing note by depth, examples, simplicity, or stack-specific focus.	Updated section or full note.
Question Generation Prompt	Generate interview questions from note or weak concepts.	Questions with difficulty, expected concepts, and source section.
Answer Evaluation Prompt	Evaluate technical correctness, missing concepts, and answer structure.	Scores, feedback, better answer, next questions.
English Feedback Prompt	Detect grammar, naturalness, clarity, and save-worthy English issues.	Short English note and weak English topics.
Weakness Mapping Prompt	Map answer failures to technical notes and concepts.	Weak concepts and recommended notes.
Learning Recommendation Prompt	Rank next topics and practice sessions.	Prioritized learning plan.

AI Reliability Concern	Mitigation
Hallucinated technical facts	Use strict structured output, validate against known topic taxonomy, allow user edits.
Too long notes	Use required sections and section-level regeneration.
Noisy English notes	Save only repeated or interview-impacting issues.
High AI cost	Queue long jobs, cache note generations, summarize prior context, limit token usage.
Inconsistent output	Use schema validation and prompt versioning.

17. Metrics and Success Criteria

Metric	Definition	Why It Matters
North Star Metric	Weekly active interview learning minutes across notebook and mock practice.	Measures meaningful preparation, not passive signups.
Note-to-practice conversion	Percentage of generated notes that lead to interview practice.	Validates notebook-interview connection.
Practice sessions per user	Average weekly mock interview sessions.	Measures engagement.
Weak concept improvement	Reduction in weakness score after repeated practice.	Measures learning effectiveness.
English issue recurrence	Frequency of repeated grammar/naturalness mistakes.	Measures English improvement.
Recommendation acceptance	User starts recommended note/question/topic.	Measures recommendation quality.
Retention	D7, D30 active users.	Measures long-term utility.
Paid conversion	Free-to-pro upgrade rate.	Measures business value.

18. Monetization Strategy

Plan	Features
Free	Limited notes, limited interview sessions, basic feedback, limited English notes.
Pro	Unlimited or higher-limit notes, mock interviews, advanced feedback, English Notebook, learning recommendations, resume analyzer.
Premium	Voice mode, company simulations, advanced system design practice, exportable reports.
Team / Bootcamp	Cohort management, shared templates, progress analytics, admin dashboard.

19. Roadmap

Phase	Scope
Phase 1 - MVP	Onboarding, profile defaults, Smart Technical Notebook, text mock interview, English Feedback Notebook, recommendations, dashboard.
Phase 2 - Voice and Resume	Speech-to-text, voice answers, pronunciation/naturalness, resume analyzer, stronger dashboard.
Phase 3 - Advanced Practice	Company simulations, system design mode, coding + explanation mode, spaced repetition.
Phase 4 - Scale and Teams	Team workspace, public note templates, bootcamp analytics, enterprise controls.

20. Risks, Tradeoffs, and Mitigations

Risk	Impact	Mitigation
AI cost grows quickly	Margins suffer as users generate long notes and feedback.	Use quotas, caching, summaries, prompt compression, async jobs.
Users generate notes but do not practice	Product becomes passive note app.	Strong CTAs from note detail to practice questions.
Too much feedback overwhelms users	Users abandon after long AI output.	Prioritize short feedback, progressive disclosure, summaries first.
English notes become noisy	Notebook loses value.	Save only repeated or interview-impacting issues.
AI feedback is inconsistent	User trust decreases.	Use rubrics, structured outputs, prompt versioning, calibration tests.
Privacy concerns with resumes/audio	Legal and trust issues.	Encryption, delete controls, data retention settings, clear consent.

21. Security, Privacy, and Compliance

- Encrypt sensitive data at rest and in transit.
- Store resumes and audio files with signed URLs and short-lived access.
- Allow users to delete notes, interview history, resumes, and audio.
- Avoid using user content for model training unless explicitly opted in.
- Implement role-based access control before team/bootcamp features.
- Rate-limit AI endpoints and protect against prompt injection in uploaded notes/resumes.
- Maintain audit logs for sensitive data access.
- Prepare GDPR-style export/delete flows for international users.

22. MVP Acceptance Criteria

Feature	Acceptance Criteria
Onboarding	A new user can complete profile setup and see defaults reused in notebook and interview modules.
Settings	User can adjust role, level, tech stack, English level, and output preferences.
Create Note	User can enter topic and rough notes without selecting role/depth repeatedly.
Generate Technical Note	AI returns the required standardized note structure.
Generate Questions from Note	System creates questions linked to note sections and expected concepts.
Mock Interview	User can answer by text and receive technical + English feedback.
English Note	System saves a short English note for important interview-impacting issues.
Recommendation	System recommends next question, technical topic, and English topic after an answer.
Dashboard	User can see weak technical concepts, weak English

	topics, and next actions.

23. Example End-to-End Scenario

12. During onboarding, the user selects Fullstack Developer, current level Mid, target level Senior, English level Intermediate, and stack React, Next.js, NestJS, PostgreSQL, Redis.
13. The user creates a note titled JWT Authentication and pastes rough notes.
14. The app generates a senior-level technical note using TypeScript, NestJS, SQL, and production tradeoffs.
15. The user clicks Generate Interview Questions from this note.
16. The AI asks: How would you revoke JWTs in a distributed system?
17. The user answers with incomplete revocation details and grammar mistakes.
18. The system saves technical weak concepts: refresh token rotation, Redis blacklist, distributed logout.
19. The system saves an English note: passive voice and present simple correction, with short practice patterns.
20. The system recommends the next technical note: Redis Token Blacklist Strategy.
21. The system recommends the next English topic: Passive Voice in technical explanations.

24. Product Positioning

DevInterview AI should be positioned as a technical interview preparation workspace for developers that combines senior-level theory notes, AI mock interviews, English speaking and writing feedback, and personalized learning recommendations.

Short Positioning

Notion + LeetCode + AI Interview Coach for software developers preparing for interviews in English.